FastAPI Learn Advanced User Guide

Settings and Environment Variables

In many cases your application could need some external settings or configurations, for example secret keys, database credentials, credentials for email services, etc.

Most of these settings are variable (can change), like database URLs. And many could be sensitive, like secrets.

For this reason it's common to provide them in environment variables that are read by the application.

> **Tip**
>
> To understand environment variables you can read Environment Variables.

Types and validation

These environment variables can only handle text strings, as they are external to Python and have to be compatible with other programs and the rest of the system (and even with different operating systems, as Linux, Windows, macOS).

That means that any value read in Python from an environment variable will be a `str`, and any conversion to a different type or any validation has to be done in code.

Pydantic `Settings`

Fortunately, Pydantic provides a great utility to handle these settings coming from environment variables with Pydantic: Settings management.

Install `pydantic-settings`

First, make sure you create your virtual environment, activate it, and then install the `pydantic-settings` package:

```bash
fast →
$ pip instal
```

It also comes included when you install the `all` extras with:

```bash
fast →
```

```
$ pip instal
```

Create the `Settings` object

Import `BaseSettings` from Pydantic and create a sub-class, very much like with a Pydantic model.

The same way as with Pydantic models, you declare class attributes with type annotations, and possibly default values.

You can use all the same validation features and tools you use for Pydantic models, like different data types and additional validations with `Field()`.

Python 3.10+

```python
from fastapi import FastAPI
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50


settings = Settings()
app = FastAPI()


@app.get("/info")
async def info():
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Tip

If you want something quick to copy and paste, don't use this example, use the last one below.

Then, when you create an instance of that `Settings` class (in this case, in the `settings` object), Pydantic will read the environment variables in a case-insensitive way, so, an upper-case variable `APP_NAME` will still be read for the attribute `app_name`.

Next it will convert and validate the data. So, when you use that `settings` object, you will have data of the types you declared (e.g. `items_per_user` will be an `int`).

Use the `settings`

Then you can use the new `settings` object in your application:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic_settings import BaseSettings
```

```python
class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50


settings = Settings()
app = FastAPI()


@app.get("/info")
async def info():
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Run the server

Next, you would run the server passing the configurations as environment variables, for example you could set an `ADMIN_EMAIL` and `APP_NAME` with:

```bash
```

> **Tip**
>
> To set multiple env vars for a single command just separate them with a space, and put them all before the command.

And then the `admin_email` setting would be set to `"deadpool@example.com"`.

The `app_name` would be `"ChimichangApp"`.

And the `items_per_user` would keep its default value of `50`.

Settings in another module

You could put those settings in another module file as you saw in Bigger Applications - Multiple Files.

For example, you could have a file `config.py` with:

Python 3.10+

```python
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
```

```
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50


settings = Settings()
```

And then use it in a file `main.py`:

Python 3.10+

```python
from fastapi import FastAPI
from .config import settings

app = FastAPI()


@app.get("/info")
async def info():
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Tip

You would also need a file `__init__.py` as you saw in Bigger Applications - Multiple Files.

Settings in a dependency

In some occasions it might be useful to provide the settings from a dependency, instead of having a global object with `settings` that is used everywhere.

This could be especially useful during testing, as it's very easy to override a dependency with your own custom settings.

The config file

Coming from the previous example, your `config.py` file could look like:

Python 3.10+

```python
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50
```

Notice that now we don't create a default instance `settings = Settings()`.

The main app file

Now we create a dependency that returns a new `config.Settings()`.

Python 3.10+

```python
from functools import lru_cache
from typing import Annotated

from fastapi import Depends, FastAPI

from .config import Settings

app = FastAPI()


@lru_cache
def get_settings():
    return Settings()


@app.get("/info")
async def info(settings: Annotated[Settings, Depends(get_settings)]):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from functools import lru_cache

from fastapi import Depends, FastAPI

from .config import Settings

app = FastAPI()


@lru_cache
def get_settings():
    return Settings()


@app.get("/info")
async def info(settings: Settings = Depends(get_settings)):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Tip

We'll discuss the `@lru_cache` in a bit.

For now you can assume `get_settings()` is a normal function.

And then we can require it from the *path operation function* as a dependency and use it anywhere we need it.

Python 3.10+

```python
from functools import lru_cache
from typing import Annotated

from fastapi import Depends, FastAPI

from .config import Settings

app = FastAPI()


@lru_cache
def get_settings():
    return Settings()


@app.get("/info")
async def info(settings: Annotated[Settings, Depends(get_settings)]):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from functools import lru_cache

from fastapi import Depends, FastAPI

from .config import Settings

app = FastAPI()


@lru_cache
def get_settings():
    return Settings()


@app.get("/info")
async def info(settings: Settings = Depends(get_settings)):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Settings and testing

Then it would be very easy to provide a different settings object during testing by creating a dependency override for `get_settings`:

Python 3.10+

```python
from fastapi.testclient import TestClient
```

```python
from .config import Settings
from .main import app, get_settings

client = TestClient(app)


def get_settings_override():
    return Settings(admin_email="testing_admin@example.com")


app.dependency_overrides[get_settings] = get_settings_override


def test_app():
    response = client.get("/info")
    data = response.json()
    assert data == {
        "app_name": "Awesome API",
        "admin_email": "testing_admin@example.com",
        "items_per_user": 50,
    }
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi.testclient import TestClient

from .config import Settings
from .main import app, get_settings

client = TestClient(app)


def get_settings_override():
    return Settings(admin_email="testing_admin@example.com")


app.dependency_overrides[get_settings] = get_settings_override


def test_app():
    response = client.get("/info")
    data = response.json()
    assert data == {
        "app_name": "Awesome API",
        "admin_email": "testing_admin@example.com",
        "items_per_user": 50,
    }
```

In the dependency override we set a new value for the `admin_email` when creating the new `Settings` object, and then we return that new object.

Then we can test that it is used.

Reading a `.env` file

If you have many settings that possibly change a lot, maybe in different environments, it might be useful to put them on a file and then read them from it as if they were environment variables.

This practice is common enough that it has a name, these environment variables are commonly placed in a file `.env`, and the file is called a "dotenv".

> **Tip**
>
> A file starting with a dot (`.`) is a hidden file in Unix-like systems, like Linux and macOS.
>
> But a dotenv file doesn't really have to have that exact filename.

Pydantic has support for reading from these types of files using an external library. You can read more at Pydantic Settings: Dotenv (.env) support.

> **Tip**
>
> For this to work, you need to `pip install python-dotenv`.

The `.env` file

You could have a `.env` file with:

```
ADMIN_EMAIL="deadpool@example.com"
APP_NAME="ChimichangApp"
```

Read settings from `.env`

And then update your `config.py` with:

Python 3.10+

```python
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

    model_config = SettingsConfigDict(env_file=".env")
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "Awesome API"
    admin_email: str
    items_per_user: int = 50

    model_config = SettingsConfigDict(env_file=".env")
```

Tip

The `model_config` attribute is used just for Pydantic configuration. You can read more at Pydantic: Concepts: Configuration.

Here we define the config `env_file` inside of your Pydantic `Settings` class, and set the value to the filename with the dotenv file we want to use.

Creating the `Settings` only once with `lru_cache`

Reading a file from disk is normally a costly (slow) operation, so you probably want to do it only once and then reuse the same settings object, instead of reading it for each request.

But every time we do:

```
Settings()
```

a new `Settings` object would be created, and at creation it would read the `.env` file again.

If the dependency function was just like:

```
def get_settings():
    return Settings()
```

we would create that object for each request, and we would be reading the `.env` file for each request. ⚠️

But as we are using the `@lru_cache` decorator on top, the `Settings` object will be created only once, the first time it's called. ✔️

Python 3.10+

```
from functools import lru_cache
from typing import Annotated

from fastapi import Depends, FastAPI

from . import config
```

```
app = FastAPI()


@lru_cache
def get_settings():
    return config.Settings()


@app.get("/info")
async def info(settings: Annotated[config.Settings, Depends(get_settings)]):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from functools import lru_cache

from fastapi import Depends, FastAPI

from . import config

app = FastAPI()


@lru_cache
def get_settings():
    return config.Settings()


@app.get("/info")
async def info(settings: config.Settings = Depends(get_settings)):
    return {
        "app_name": settings.app_name,
        "admin_email": settings.admin_email,
        "items_per_user": settings.items_per_user,
    }
```

Then for any subsequent call of `get_settings()` in the dependencies for the next requests, instead of executing the internal code of `get_settings()` and creating a new `Settings` object, it will return the same object that was returned on the first call, again and again.

`lru_cache` Technical Details

`@lru_cache` modifies the function it decorates to return the same value that was returned the first time, instead of computing it again, executing the code of the function every time.

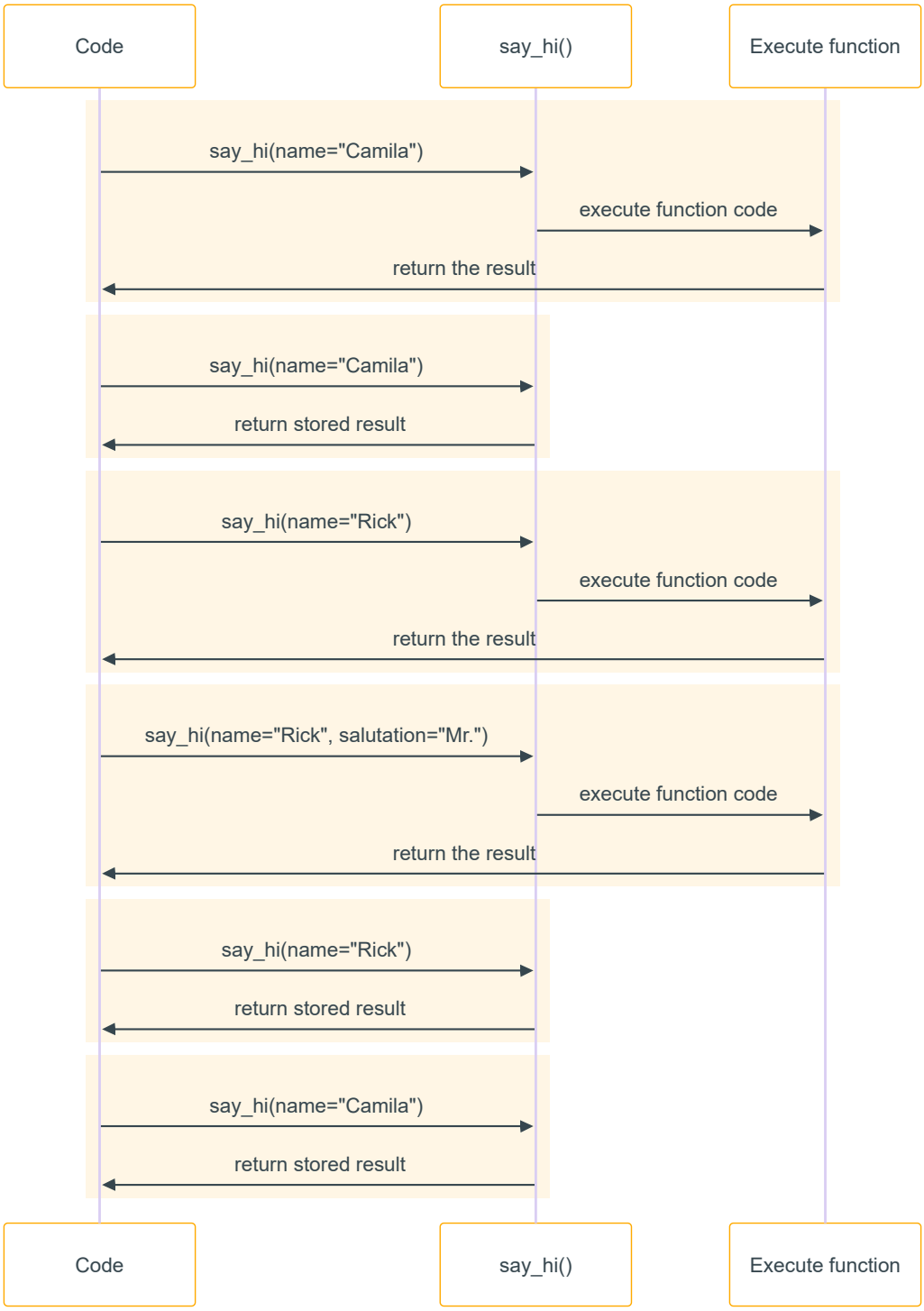
So, the function below it will be executed once for each combination of arguments. And then the values returned by each of those combinations of arguments will be used again and again whenever the function is called with exactly the same combination of arguments.

For example, if you have a function:

```
@lru_cache
```

```python
def say_hi(name: str, salutation: str = "Ms."):
    return f"Hello {salutation} {name}"
```

your program could execute like this:



In the case of our dependency `get_settings()`, the function doesn't even take any arguments, so it always returns the same value.

That way, it behaves almost as if it was just a global variable. But as it uses a dependency function, then we can override it easily for testing.

`@lru_cache` is part of `functools` which is part of Python's standard library, you can read more about it in the Python docs for `@lru_cache`.

Recap

You can use Pydantic Settings to handle the settings or configurations for your application, with all the power of Pydantic models.

- By using a dependency you can simplify testing.
- You can use `.env` files with it.
- Using `@lru_cache` lets you avoid reading the dotenv file again and again for each request, while allowing you to override it during testing.